Chapitre 0 — Avant tout | Partie C — Déplacer une lettre | leçon 0.33.3 sur 153 | difficulté 0 / 10

0.33.3. La fonction carre() — tourner en carré

#include <carre> : carre(x, y, tuile, taille, sens, vitesse, tours) fait tourner la tuile en carré autour de (x, y).

1. Ce qu'il faut comprendre

Ce qu'elle fait : `carre(x, y, tuile, taille, sens, vitesse, tours)` fait faire à la tuile des **tours en carré** autour de la case (x, y). Taille 1 : un carré de 3 × 3 ; taille 2 : 5 × 5. Elle part du centre et y revient.

La ligne à écrire : `#include <carre>`.

Ses arguments : le centre, la tuile, la taille, le sens (1 : aiguilles d'une montre, -1 : l'autre), la vitesse en ms par pas (en clair), le nombre de tours.

Ce qu'elle coûte : environ **500 octets**, plus la police entière.

Essaie : `carre(10, 8, ALPHABET[0], 3, -1, 100, 2);`.

> **On retrouve carre dans 63 leçons**
>
> 0.34, 0.34.1, 0.34.2, 0.35, 0.35.1, 0.35.2, 0.36, 0.36.1, 0.36.2, 0.37, 0.37.1, 0.37.2, 0.38, 0.38.1, 0.38.2, 0.39, 0.39.1, 0.39.2, 0.40, 0.40.1, 0.40.2, 0.41, 0.41.1, 0.41.2, 0.42, 0.42.1, 0.42.2, 0.43, 0.43.1, 0.43.2, 0.44, 0.44.1, 0.44.2, 0.45, 0.45.1, 0.45.2, 0.46, 0.46.1, 0.46.2, 0.47, 0.47.1, 0.47.2, 0.48, 0.48.1, 0.48.2, 0.49, 0.49.1, 0.49.2, 0.50, 0.50.1, 0.50.2, 0.51, 0.51.1, 0.51.2, 0.52, 0.52.1, 0.52.2, 0.53, 0.53.1, 0.53.2, 0.54, 0.54.1, 0.54.2

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// ---- #include <carre> : tourner en carré ----`	Un commentaire : une phrase pour celui qui lit (« ---- #include <carre> : tourner en carré ---- »). La console l'ignore.
`// Un C qui tourne autour de la case (15, 4), en haut à droite.`	Un commentaire : une phrase pour celui qui lit (« Un C qui tourne autour de la case (15, 4), en haut à droite. »). La console l'ignore.
`// (Le A au milieu, ce sera la leçon suivante.)`	Un commentaire : une phrase pour celui qui lit (« (Le A au milieu, ce sera la leçon suivante.) »). La console l'ignore.
`#include <carre>  // carre() : des tours en carré`	Demande à la console « carre » (ce que c'est : une tuile qui tourne en carré). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « carre() : des tours en carré ».)
`#include <ALPHABET>`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer.
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.

`carre(15, 4, ALPHABET[2], 1, 1, 250, 1);  // centre (15, 4), taille 1, un tour`	Appel de la fonction carre de la console, avec les arguments 15, 4, ALPHABET[2], 1, 1, 250, 1. Ce qu'elle fait : un carré parfait AUTOUR de (x, y) : taille 1 fait 3 × 3, taille 2 fait 5 × 5 (2 × taille + 1). sens 1 : aiguilles d'une montre, -1 : l'autre sens. vitesse : millisecondes par pas, écrite en clair (250). La lettre part du centre, fait ses tours et y revient. Ou, les sept réglages sous un nom : Carre ronde = { 10, 8, ALPHABET[0], 2, 1, 250, 3 }; carre(ronde);. (Le commentaire le dit : « centre (15, 4), taille 1, un tour ».)
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1349 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  carre()         440 o  ← main()  — une fonction fournie par le langage
  main()           38 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EffacerCases  — attend de pouvoir écrire une case
  AdresseCase      10 o  ← poser(), effacer()  — trouve l'adresse d'une case de l'écran
  EffacerCases     12 o  ← effacer()  — efface des cases
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette,
DonneesTransfert, Notes
Fonctions du langage appelées :
  poser()   × 4    52 o sur place, dans carre()  + AdresseCase (10 o)
  effacer() × 3    45 o sur place, dans carre()  + AdresseCase (10 o), EffacerCases (12 o)
  image()   × 4    12 o sur place, dans carre(), main()  + AttendreImage (28 o)
Tes fonctions appelées :
  carre()   × 1    31 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <carre> : tourner en carré ----

// Un C qui tourne autour de la case (15, 4), en haut à droite.
// (Le A au milieu, ce sera la leçon suivante.)

#include <carre>      // carre() : des tours en carré
#include <ALPHABET>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

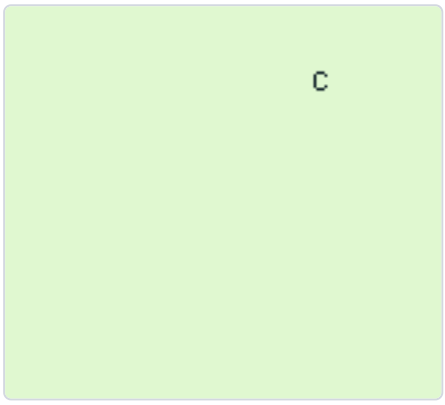
```
int main() {
  carre(15, 4, ALPHABET[2], 1, 1, 250, 1);   // centre (15, 4), taille 1, un tour

  while (true) {
    image();
  }
}
```

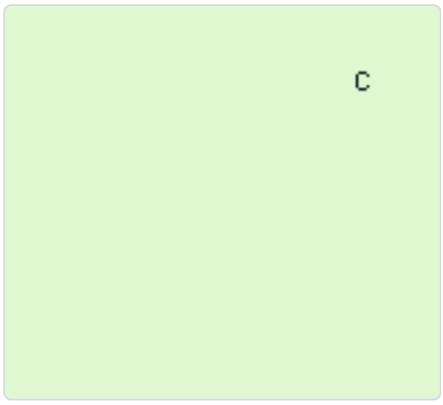
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

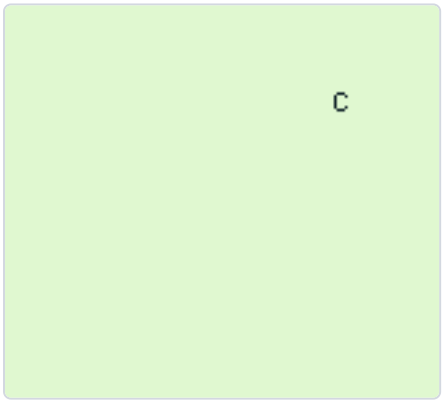
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

> Un C qui fait un tour en carré autour de la case (15, 4), puis s'y arrête.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ les quatre coins, dans le sens des aiguilles (14,3 16,3 16,5 14,5)

✓ le C revient au centre

Ensuite : **0.34. Un carré autour d'une lettre, avec carre** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++